

Lab 03 — Data Structures, Classes and Generators

DSA27 Lab Manual · Week 3 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 3

Contents

Part 1 — More on lists	2
Part 2 — Comprehensions	5
Part 3 — Tuples	7
Part 4 — Sets	8
Part 5 — Dictionaries	9
Part 6 — Looping techniques	11
Part 7 — Classes	12
Part 8 — Iterators and generators	17
Part 9 — Exercises	19
Part 10 — Bridge to this week's lecture: recursion	21
Part 11 — Take-home practice (not graded)	22
Summary	22
Answers to the checkpoints	23

How to use this lab. Same routine: type every example, predict at each **Checkpoint** (answers at the end), then make `pytest tests/test_Lab03.py` pass. This is the longest of the three labs and the most important: from Week 4 on, every structure you build is a **class**, and the lists and dictionaries of this lab are exactly what you will learn to build yourself — inside `dsa/` you may not use them for storage (Lecture 02).

Duration	One 2-hour lab session, plus about 4 hours at home
You will write	<code>labs/lab03.py</code> — 9 functions and one class, <code>Bag</code>
Graded by	<code>tests/test_lab03.py</code>

Connects to

Lecture 03 — Recursion (Part 11), and every week after it

What you will be able to do

1. use every list method, and say which ones are $O(1)$ and which are $O(n)$;
2. use a list as a stack, and know why it makes a bad queue;
3. write list, set and dictionary comprehensions;
4. choose between a list, a tuple, a set and a dictionary for a job — and justify it by cost;
5. write a class with `__init__`, methods and special methods such as `__len__`, `__iter__`, `__eq__` and `__repr__`;
6. write a generator with `yield`, and explain what for does behind the scenes;
7. trace a recursive function on the call stack.

Part 1 — More on lists**1.1 All the list methods**

Method	What it does	Cost
<code>a.append(x)</code>	add <code>x</code> at the end	$O(1)^*$
<code>a.extend(iterable)</code>	add every item of <code>iterable</code> at the end	$O(k)$
<code>a.insert(i, x)</code>	insert <code>x</code> before position <code>i</code>	$O(n)$
<code>a.remove(x)</code>	remove the first item equal to <code>x</code> ; <code>ValueError</code> if none	$O(n)$
<code>a.pop()</code>	remove and return the last item	$O(1)$
<code>a.pop(i)</code>	remove and return the item at <code>i</code>	$O(n)$
<code>a.clear()</code>	remove everything	$O(n)$
<code>a.index(x)</code>	position of the first <code>x</code> ; <code>ValueError</code> if none	$O(n)$
<code>a.count(x)</code>	how many times <code>x</code> appears	$O(n)$
<code>a.sort()</code>	sort in place	$O(n \log n)$
<code>a.reverse()</code>	reverse in place	$O(n)$
<code>a.copy()</code>	a shallow copy, same as <code>a[:]</code>	$O(n)$
<code>x in a</code>	membership test	$O(n)$
<code>a[i], a[i] = x, len(a)</code>	index, assign, length	$O(1)$

* *amortised* — usually instant, occasionally slow when the list has to grow. You will build this yourself in Week 4 and see why the average stays $O(1)$.

Where do those costs come from? A Python list is an **array of references** stored side by side in memory. Reaching `a[i]` is one address calculation — $O(1)$. But inserting at the front means shifting every other reference one place to the right — $O(n)$. Looking for a value means checking items one by one — $O(n)$. Keep that picture; the whole of Week 4 is built on it.

```
>>> fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana']
>>> fruits.count('apple')
2
>>> fruits.index('banana')
3
>>> fruits.index('banana', 4)      # search starting from position 4
6
>>> fruits.reverse()
>>> fruits
['banana', 'apple', 'kiwi', 'banana', 'pear', 'apple', 'orange']
>>> fruits.append('grape')
>>> fruits.sort()
>>> fruits
['apple', 'apple', 'banana', 'banana', 'grape', 'kiwi', 'orange', 'pear']
>>> fruits.pop()
'pear'
```

1.2 sort() versus sorted()

`a.sort()` changes the list and returns **None**. `sorted(a)` leaves `a` alone and returns a **new** list. `sorted` works on any iterable; `sort` only on lists.

```
>>> marks = [72, 95, 60]
>>> result = marks.sort()
>>> print(result)      # the classic mistake
None
>>> sorted('python')
['h', 'n', 'o', 'p', 't', 'y']
```

Both take `key=` (a function applied to each item, whose result is compared) and `reverse=True`:

```
>>> words = ['banana', 'Kiwi', 'apple', 'Cherry']
>>> sorted(words)      # capitals sort before lower case
['Cherry', 'Kiwi', 'apple', 'banana']
>>> sorted(words, key=str.lower)
['apple', 'banana', 'Cherry', 'Kiwi']
>>> sorted(words, key=len, reverse=True)
['banana', 'Cherry', 'apple', 'Kiwi']
```

Sorting by several things — the key returns a *tuple*, and tuples compare item by item (Part 5):

```
>>> students = [('mona', 88), ('ali', 95), ('omar', 88)]
>>> sorted(students, key=lambda s: (-s[1], s[0])) # mark high->low, then name
```

```
[('ali', 95), ('mona', 88), ('omar', 88)]
```

Python's sort is **stable**: items that compare equal keep their original order. You will see why that property matters when you write sorts in Weeks 9–10.

Items must be comparable with each other: `sorted([3, 'a'])` is a `TypeError`.

1.3 Lists as stacks

A **stack** is last-in, first-out (LIFO) — like a pile of plates. With a list, push is `append` and pop is `pop()`, both $O(1)$ at the **end**:

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack.pop()
6
>>> stack
[3, 4, 5]
```

In Week 6 you write a `Stack` class with exactly this behaviour — on top of your own `Dynami- cArray`, not a list.

1.4 Lists as queues — and why not

A **queue** is first-in, first-out (FIFO). You *can* use a list — `append` to join, `pop(0)` to leave — but `pop(0)` shifts every remaining item: $O(n)$ per operation. For a real queue use `collections.deque`, which is $O(1)$ at both ends:

```
>>> from collections import deque
>>> queue = deque(['Eric', 'John', 'Michael'])
>>> queue.append('Terry')           # Terry arrives
>>> queue.append('Graham')         # Graham arrives
>>> queue.popleft()                # the first to arrive leaves
'Eric'
>>> queue.popleft()
'John'
>>> queue
deque(['Michael', 'Terry', 'Graham'])
```

In Week 7 you will **measure** the difference between these two, and then build the $O(1)$ version yourself as a ring buffer.

1.5 The del statement

del removes by **position** or **slice** (where remove removes by value):

```
>>> a = [-1, 1, 66.25, 333, 333, 1234.5]
>>> del a[0]
>>> a
[1, 66.25, 333, 333, 1234.5]
>>> del a[2:4]
>>> a
[1, 66.25, 1234.5]
>>> del a[:]
>>> a
[]
>>> del a           # deletes the NAME; using `a` now is a NameError
```

Part 2 — Comprehensions

2.1 List comprehensions

The loop-and-append pattern is so common that Python has a short form for it:

```
squares = []
for x in range(10):
    squares.append(x ** 2)

squares = [x ** 2 for x in range(10)]           # same result
```

A comprehension is brackets containing an expression, then a for, then any number of further for or if clauses. The clauses read in the same order as the nested loops they replace:

```
>>> [x for x in range(20) if x % 3 == 0]
[0, 3, 6, 9, 12, 15, 18]
>>> [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

That second one is exactly:

```
combs = []
for x in [1, 2, 3]:
    for y in [3, 1, 4]:
        if x != y:
            combs.append((x, y))
```

More examples:

```
>>> vec = [-4, -2, 0, 2, 4]
>>> [x * 2 for x in vec]
```

```
[-8, -4, 0, 4, 8]
>>> [abs(x) for x in vec]
[4, 2, 0, 2, 4]
>>> fresh = [' banana', ' loganberry ', 'passion fruit ']
>>> [w.strip() for w in fresh]
['banana', 'loganberry', 'passion fruit']
>>> [(x, x ** 2) for x in range(4)]      # a tuple needs its parentheses
[(0, 0), (1, 1), (2, 4), (3, 9)]
>>> vec2 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
>>> [num for elem in vec2 for num in elem] # flatten one level
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> ['even' if x % 2 == 0 else 'odd' for x in range(4)] # if-else BEFORE the for
['even', 'odd', 'even', 'odd']
```

Note the two different ifs: ... for x in xs if cond **filters** items; a if cond else b for x in xs **chooses** a value for every item.

2.2 Nested comprehensions

The expression at the front can itself be a comprehension. This 3×4 matrix is a list of three rows:

```
>>> matrix = [
...     [1, 2, 3, 4],
...     [5, 6, 7, 8],
...     [9, 10, 11, 12],
... ]
>>> [[row[i] for row in matrix] for i in range(4)]      # transpose
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

Read it from the outside in: *for each column index i, build the list of row[i] for every row.* (The built-in way is `list(zip(*matrix))`, which gives tuples.)

This is also the correct way to make the grid that went wrong in Lab 01:

```
>>> board = [[0] * 3 for _ in range(3)]      # three SEPARATE inner lists
>>> board[0][0] = 'X'
>>> board
[['X', 0, 0], [0, 0, 0], [0, 0, 0]]
```

`_` is the conventional name for a loop variable you do not use.

When not to. A comprehension should fit on a line or two and be read at a glance. If it needs three conditions and a nested loop, write the loop.

Checkpoint 1. Predict:

- (a) `[c.upper() for c in 'abc' if c != 'b']`
- (b) `[i * j for i in range(1, 3) for j in range(1, 3)]`
- (c) `[[i * j for j in range(1, 3)] for i in range(1, 3)]`

(d) `[len(w) for w in 'to be or not'.split() if len(w) > 2]`

Part 3 — Tuples

A **tuple** is a sequence of values separated by commas, usually in parentheses. It indexes and slices like a list, but it is **immutable**:

```
>>> t = 12345, 54321, 'hello!'
>>> t[0]
12345
>>> t
(12345, 54321, 'hello!')
>>> u = t, (1, 2, 3)           # tuples can nest
>>> u
((12345, 54321, 'hello!'), (1, 2, 3))
>>> t[0] = 88888
Traceback (most recent call last):
...
TypeError: 'tuple' object does not support item assignment
```

A tuple cannot change, but an object *inside* it can:

```
>>> v = ([1, 2, 3], [3, 2, 1])
>>> v[0].append(4)
>>> v
([1, 2, 3, 4], [3, 2, 1])
```

Empty and one-item tuples have awkward syntax — it is the comma that makes a tuple, not the parentheses:

```
>>> empty = ()
>>> singleton = 'hello',      # note the trailing comma
>>> len(empty), len(singleton)
(0, 1)
>>> type(('hello'))           # just a string in parentheses
<class 'str'>
```

Packing and unpacking. `t = 1, 2, 3` packs three values into a tuple. `x, y, z = t` unpacks them — the number of names must match. This is what the multiple assignment in Lab 01 was all along, and what happens when a function “returns two values”. A starred name takes the rest:

```
>>> first, *rest = [1, 2, 3, 4]
>>> first, rest
(1, [2, 3, 4])
>>> for name, mark in [('ali', 95), ('mona', 88)]: # unpacking in a for
...     print(name, mark)
```

List or tuple? Use a **list** for a variable-length collection of similar things (the students in a class).

Use a **tuple** for a fixed group of related values, often of different types (one student's (name, id, mark)). And only immutable things can be dictionary keys or set members, so a tuple can be a key where a list cannot.

Part 4 — Sets

A **set** is an unordered collection with **no duplicates**. Its superpower is membership testing in **O(1) on average** — against O(n) for a list — because it is built on a hash table, which you will build yourself in Week 13.

```
>>> basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
>>> basket                                     # duplicates gone; order not guaranteed
{'orange', 'banana', 'pear', 'apple'}
>>> 'orange' in basket                         # fast
True
>>> 'crabgrass' in basket
False
```

{ } is an **empty dictionary**, not an empty set. Use set():

```
>>> type({}), type(set())
(<class 'dict'>, <class 'set'>)
```

Set algebra:

```
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a                                     # unique letters in a
{'a', 'r', 'b', 'c', 'd'}
>>> a - b                                 # in a but not in b
{'r', 'd', 'b'}
>>> a | b                                 # in a or b or both
{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
>>> a & b                                 # in both
{'a', 'c'}
>>> a ^ b                                 # in a or b but not both
{'r', 'd', 'b', 'm', 'z', 'l'}
```

(The order the REPL prints a set in may differ on your machine. That is the point: a set has no order.)

Methods: s.add(x), s.remove(x) (KeyError if absent), s.discard(x) (no error), s.pop(), len(s). Set comprehensions work too:

```
>>> {x for x in 'abracadabra' if x not in 'abc'}
{'r', 'd'}
```

Set members must be **hashable** — in practice, immutable: numbers, strings, tuples of those. A

list cannot go into a set.

The pattern you will use constantly — “have I seen this before?”:

```
seen = set()
for item in items:
    if item in seen:
        print('duplicate:', item)
    seen.add(item)
```

With a list for seen, that loop is $O(n^2)$. With a set it is $O(n)$. Exercise 1 has a test that only an $O(n)$ solution passes in reasonable time.

Part 5 — Dictionaries

A **dictionary** maps **keys** to **values**. Keys are unique and hashable; values can be anything. Looking up, adding, changing and removing by key are all $O(1)$ on average — dictionaries are hash tables too.

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127           # add
>>> tel
{'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel['jack']                   # Look up
4098
>>> del tel['sape']               # remove
>>> tel['irv'] = 4127
>>> tel
{'jack': 4098, 'guido': 4127, 'irv': 4127}
>>> list(tel)                    # the keys, in insertion order
['jack', 'guido', 'irv']
>>> sorted(tel)
['guido', 'irv', 'jack']
>>> 'guido' in tel                # `in` checks KEYS
True
>>> 'jack' not in tel
False
>>> tel['nobody']
Traceback (most recent call last):
...
KeyError: 'nobody'
```

Dictionaries **remember insertion order** (guaranteed since Python 3.7). Assigning to an existing key replaces its value and keeps its position.

Other ways to build one:

```
>>> dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
{'sape': 4139, 'guido': 4127, 'jack': 4098}
>>> dict(sape=4139, guido=4127, jack=4098)      # keys that are simple names
{'sape': 4139, 'guido': 4127, 'jack': 4098}
>>> {x: x ** 2 for x in (2, 4, 6)}              # dict comprehension
{2: 4, 4: 16, 6: 36}
>>> dict(zip(['a', 'b'], [1, 2]))
{'a': 1, 'b': 2}
```

5.1 Reading safely

```
>>> tel.get('nobody')          # None instead of KeyError
>>> tel.get('nobody', 0)      # or a default of your choice
0
>>> tel.keys(), tel.values(), tel.items()
(dict_keys(['jack', 'guido', 'irv']), dict_values([4098, 4127, 4127]),
 dict_items([('jack', 4098), ('guido', 4127), ('irv', 4127)]))
>>> tel.pop('irv')            # remove and return
4127
```

5.2 The counting pattern

Counting things is the most common dictionary job, and it has three idioms:

```
words = 'the cat and the hat and the bat'.split()
```

1. get with a default

```
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1
```

2. setdefault – for grouping, when the value is a list

```
by_letter = {}
for w in words:
    by_letter.setdefault(w[0], []).append(w)
```

3. the library does it for you

```
from collections import Counter
Counter(words).most_common(2)      # [('the', 3), ('and', 2)]
```

```
>>> counts
{'the': 3, 'cat': 1, 'and': 2, 'hat': 1, 'bat': 1}
>>> by_letter
{'t': ['the', 'the', 'the'], 'c': ['cat'], 'a': ['and', 'and'], 'h': ['hat'], 'b': ['bat']}
```

`setdefault(key, default)` returns the value for `key`, first storing `default` there if the key

was missing. `collections.defaultdict(list)` does the same job automatically. Know all three; in the exercises, write the first two yourself.

Part 6 — Looping techniques

```
>>> knights = {'gallahad': 'the pure', 'robin': 'the brave'}
>>> for k, v in knights.items():           # key and value together
...     print(k, v)
...
gallahad the pure
robin the brave
>>> for i, v in enumerate(['tic', 'tac', 'toe']):
...     print(i, v)
...
0 tic
1 tac
2 toe
>>> for i in reversed(range(1, 10, 2)):    # backwards
...     print(i, end=' ')
...
9 7 5 3 1
>>> basket = ['apple', 'orange', 'apple', 'pear', 'orange', 'banana']
>>> for f in sorted(set(basket)):           # unique, in order
...     print(f, end=' ')
...
apple banana orange pear
```

Changing a dictionary while looping over it is an error (`RuntimeError: dictionary changed size during iteration`). Loop over `list(d)` — a copy of the keys — or build a new dictionary:

```
>>> raw = [56.2, float('nan'), 51.7, 55.3, 52.5, float('nan'), 47.8]
>>> import math
>>> [value for value in raw if not math.isnan(value)]
[56.2, 51.7, 55.3, 52.5, 47.8]
```

6.1 Comparing sequences

Sequences of the same type compare **lexicographically** — like words in a dictionary: the first items are compared; if equal, the second; and so on. The shorter sequence is smaller if it runs out first:

```
(1, 2, 3) < (1, 2, 4)           # True
[1, 2, 3] < [1, 2, 4]          # True
```

```
'ABC' < 'C' < 'Pascal' < 'Python'           # True
(1, 2, 3, 4) < (1, 2, 4)                       # True
(1, 2) < (1, 2, -1)                             # True
(1, 2, 3) == (1.0, 2.0, 3.0)                   # True
(1, 2, ('aa', 'ab')) < (1, 2, ('abc', 'a'), 4) # True
```

That is why a sort key that returns a tuple sorts by several fields at once. Strings compare by character code, so all capitals come before all lower-case letters: 'Z' < 'a' is True.

6.2 Which structure?

Need	Use	Why
Ordered items, add at the end, index by position	list	O(1) index and append
A fixed record of a few values	tuple	immutable, can be a key
"Is x in here?", no duplicates	set	O(1) average membership
Look up a value by a key	dict	O(1) average lookup
Add and remove at both ends	collections.deque	O(1) at both ends
Count things	dict or Counter	

This table is the course in miniature: the same data, a different structure, a different cost. From Week 4 you stop *using* these and start *building* them.

Checkpoint 2. Predict:

```
d = {'a': 1, 'b': 2}
d['c'] = d.get('a', 0) + d.get('z', 10)
d['a'] = 5
print(List(d.items()))
print(Len({1, 1.0, True, 'one'}))
print(sorted({'b': 1, 'a': 2}, key=Lambda k: -Len(k)))
```

Part 7 — Classes

7.1 Why classes, in this course

Lecture 01 separated the **ADT** — the contract: which operations exist and what they cost — from the **data structure** that keeps the contract. A class is how Python lets you write both in one place: the methods are the contract; the attributes are the structure; and code outside uses only the methods.

dsa/stack.py is a class. So are the dynamic array, the linked list, the queues, the tree, the heap, the hash map and the graph. Every one of them has the shape you learn in this part.

7.2 A first class

```
class Dog:
    """A dog with a name and a list of tricks."""

    kind = 'canine'                # CLASS variable: shared by every Dog

    def __init__(self, name):
        self.name = name          # INSTANCE variables: one per Dog
        self.tricks = []

    def add_trick(self, trick):
        self.tricks.append(trick)

    def describe(self):
        return f'{self.name} knows {len(self.tricks)} tricks'

>>> d = Dog('Fido')              # create an instance: calls __init__
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
>>> e.add_trick('play dead')
>>> d.tricks
['roll over']
>>> d.describe()
'Fido knows 1 tricks'
>>> d.kind, e.kind              # shared
('canine', 'canine')
```

What each part means:

- `class Dog:` creates a new type. Its name is UpperCamelCase.
- `__init__` is the *initialiser*. Calling `Dog('Fido')` creates an empty object and immediately calls `__init__(that_object, 'Fido')`. Its job is to set up the object's attributes. It returns nothing.
- **self** is the object the method was called on. `d.add_trick('roll over')` is exactly `Dog.add_trick(d, 'roll over')`. The name `self` is only a convention — but never use another.
- **Attributes** are created by assigning to `self.something`. Each object has its own.
- A **method** is a function defined in the class. It always takes `self` first.

The shared-list trap. A mutable **class** variable is shared by every instance — the same mistake as a mutable default argument:

```
class BadDog:
    tricks = []                  # ONE list for ALL dogs

    def __init__(self, name):
```

```

        self.name = name

    def add_trick(self, trick):
        self.tricks.append(trick)

>>> d, e = BadDog('Fido'), BadDog('Buddy')
>>> d.add_trick('roll over')
>>> e.tricks                                # Buddy "knows" Fido's trick
['roll over']

```

Rule: **create per-object data in `__init__`**. Class variables are for constants.

7.3 Encapsulation by convention

Python has no truly private attributes. By convention, a name starting with one underscore — `self._items` — means *internal: do not touch from outside*. The `dsa/` skeletons use this everywhere:

```

class Stack:
    def __init__(self):
        self._items = []                # the data structure: private

    def push(self, value):               # the ADT: public
        self._items.append(value)

```

Code outside the class should call `push`, never `stack._items.append`. Then you can change how the stack is stored — say, to a linked list — without breaking anyone. That freedom is the whole point of separating ADT from implementation.

(This example uses a Python list only to show the idea. Inside `dsa/` a list is **not allowed** as storage: from Lecture 02, every structure keeps its data in the course `Array`, in nodes, or in a structure you built — and `dsa/stack.py` is built on your own `DynamicArray`.)

7.4 Special methods — making your class feel built in

Methods whose names start and end with double underscores (“dunder” methods) are called by Python itself when you use built-in syntax on your object:

You write	Python calls
<code>len(obj)</code>	<code>obj.__len__()</code>
<code>x in obj</code>	<code>obj.__contains__(x)</code>
<code>for x in obj</code>	<code>obj.__iter__()</code>
<code>obj[i]</code>	<code>obj.__getitem__(i)</code>
<code>obj[i] = v</code>	<code>obj.__setitem__(i, v)</code>
<code>obj == other</code>	<code>obj.__eq__(other)</code>
<code>obj < other</code>	<code>obj.__lt__(other)</code>
<code>obj + other</code>	<code>obj.__add__(other)</code>

You write	Python calls
<code>repr(obj)</code> , the REPL	<code>obj.__repr__()</code>
<code>str(obj)</code> , <code>print(obj)</code>	<code>obj.__str__()</code> (falls back to <code>__repr__</code>)
<code>if obj:</code>	<code>obj.__bool__()</code> , else <code>obj.__len__() != 0</code>

A complete worked example — a fraction that always stays in lowest terms:

```
import math

class Fraction:
    """An exact rational number numerator/denominator, in lowest terms."""

    def __init__(self, numerator, denominator=1):
        if denominator == 0:
            raise ZeroDivisionError('denominator must not be zero')
        if denominator < 0:
            # keep the sign on top
            numerator, denominator = -numerator, -denominator
        g = math.gcd(numerator, denominator)
        self._num = numerator // g
        self._den = denominator // g

    def __repr__(self):
        return f'Fraction({self._num}, {self._den})'

    def __str__(self):
        return f'{self._num}/{self._den}'

    def __eq__(self, other):
        if not isinstance(other, Fraction):
            return NotImplemented
        return self._num == other._num and self._den == other._den

    def __lt__(self, other):
        return self._num * other._den < other._num * self._den

    def __add__(self, other):
        return Fraction(self._num * other._den + other._num * self._den,
                        self._den * other._den)

    def __mul__(self, other):
        return Fraction(self._num * other._num, self._den * other._den)

>>> half = Fraction(1, 2)
>>> third = Fraction(2, 6)
>>> third
```

stored as 1/3

```

Fraction(1, 3)
>>> print(half + third)
5/6
>>> half * third
Fraction(1, 6)
>>> Fraction(3, 6) == half
True
>>> sorted([half, third, Fraction(3, 4)])    # works because of __lt__
[Fraction(1, 3), Fraction(1, 2), Fraction(3, 4)]
>>> Fraction(1, 10) + Fraction(2, 10) == Fraction(3, 10)    # exact, unlike 0.1 + 0.2
True

```

Points to notice:

- **`__repr__` should be unambiguous**, ideally code that recreates the object. It is what the REPL, the debugger and a failing pytest message show you. A good `__repr__` saves hours. `__str__` is the friendly version for users.
- **`__eq__` returns `NotImplemented`** (a special value, not the exception) when it does not know how to compare with the other type. Python then tries the other side, and finally falls back to `is`.
- **Validate in `__init__`**: an object that is created invalid stays invalid.
- Without `__eq__`, `==` means `is` — two separately created equal fractions would be “different”.
- Python’s standard library has a full version of this class: `from fractions import Fraction`. Building it yourself is the point.

7.5 Inheritance, briefly

A class can **extend** another, inheriting its methods and overriding some:

```

class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return '...'

    def introduce(self):
        return f'I am {self.name}. {self.speak()}'

class Cat(Animal):                                # Cat is-an Animal
    def speak(self):                             # override
        return 'Meow'

class Lion(Cat):
    def __init__(self, name, pride):
        super().__init__(name)                   # run the parent's __init__ first

```

```
        self.pride = pride

    def speak(self):
        return super().speak().upper() + '!'

>>> Lion('Simba', 'Pride Rock').introduce()
'I am Simba. MEOW!'
>>> isinstance(Lion('x', 'y'), Animal)
True
>>> issubclass(Cat, Lion)
False
```

introduce is defined once, in Animal, but calls whichever speak belongs to the actual object. In this course inheritance is used mainly for one thing — **your own exception types**:

```
class EmptyStackError(Exception):
    """Raised when popping or peeking an empty stack."""

def pop(stack):
    if not stack:
        raise EmptyStackError('pop from an empty stack')
    return stack.pop()
```

except EmptyStackError: then catches exactly that problem and nothing else. Since it inherits from Exception, except Exception: would also catch it.

Checkpoint 3. What is printed?

```
class Counter:
    total = 0
    def __init__(self):
        self.count = 0
    def tick(self):
        self.count += 1
        Counter.total += 1
    def __len__(self):
        return self.count

a, b = Counter(), Counter()
a.tick(); a.tick(); b.tick()
print(len(a), len(b), Counter.total, a.total, bool(Counter()))
```

Part 8 — Iterators and generators

8.1 What for really does

for x in thing: works on lists, strings, dictionaries, files, ranges. How? It asks thing for an **iterator** with iter(), then calls next() on it until the iterator raises StopIteration:

```
>>> it = iter('abc')
>>> next(it)
'a'
>>> next(it)
'b'
>>> next(it)
'c'
>>> next(it)
Traceback (most recent call last):
...
StopIteration
```

for catches that StopIteration for you and ends the loop. So to make *your* class work in a for loop, list(), sorted(), sum() and in, you give it an `__iter__` method that returns an iterator. The easy way to write one is a generator.

8.2 Generators

A **generator** is a function that uses yield instead of return. Calling it does not run it — it returns a generator object. Each next() runs the body until the next yield, hands that value out, and **pauses** there, keeping all its local variables until it is resumed:

```
def reverse(data):
    for index in range(len(data) - 1, -1, -1):
        yield data[index]
```

```
>>> for char in reverse('golf'):
...     print(char, end=' ')
...
f l o g
>>> g = reverse('ab')
>>> g
<generator object reverse at 0x...>
>>> next(g), next(g)
('b', 'a')
```

When the function body ends, the generator raises StopIteration for you.

Why generators matter:

- **Laziness.** Values are produced one at a time, when asked for. A generator over a billion items uses almost no memory — like range.
- **Easy iterators.** `__iter__` written as a generator is three lines:

```
class Countdown:
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        n = self.start
        while n > 0:
            yield n
            n -= 1
```

```
>>> list(Countdown(3))
[3, 2, 1]
>>> sum(Countdown(100))
5050
```

- **Pausing an algorithm.** The course's animation tools use this. Write a sort that `yields` the list after every comparison, and `viz.animate` can play it frame by frame (see the repository `README.md`). You will do exactly that in Week 9.

8.3 Generator expressions

A comprehension in **parentheses** instead of brackets is a generator expression: it produces values lazily instead of building a list. Use it when the values are consumed straight away by a function:

```
>>> sum(i * i for i in range(10))           # no list is ever built
285
>>> xvec, yvec = [10, 20, 30], [7, 5, 3]
>>> sum(x * y for x, y in zip(xvec, yvec))  # dot product
260
>>> max(len(w) for w in ['to', 'be', 'or', 'not'])
3
>>> any(n % 7 == 0 for n in [3, 14, 5])    # stops at the first True
True
```

Part 9 — Exercises

All in `labs/lab03.py`. Run `pytest tests/test_lab03.py -v`.

#	Name	Practises
1	<code>unique_in_order(values)</code>	a set for “seen”, a list for order — $O(n)$
2	<code>word_frequencies(text)</code>	<code>split</code> , <code>strip</code> , the counting pattern

#	Name	Practises
3	<code>top_k(frequencies, k)</code>	<code>items()</code> , sorted with a tuple key, slicing
4	<code>transpose(matrix)</code>	nested list comprehension
5	<code>invert(mapping)</code>	grouping with <code>setdefault</code> , then sorting
6	<code>common_elements(first, second)</code>	set intersection
7	<code>group_by_length(words)</code>	grouping into lists, order preserved
8	<code>Bag</code>	a class with a dict inside and eight methods
9	<code>countdown(n)</code>	a generator
10	<code>chunks(values, size)</code>	a generator with slicing

About Bag. A *bag* (or *multiset*) is an ADT: like a set, but it counts. `Bag(['a', 'b', 'a'])` holds two 'a's and one 'b'. The docstring is the full contract. Design notes:

- Keep a dictionary `self._counts` mapping each item to how many there are. Do not keep a list of all items — `count` and `in` would be $O(n)$.
- `__len__` must be $O(1)$. Recounting all the values every time is $O(k)$; keep a running total in another attribute and update it in `add` and `remove`.
- When the last copy of an item is removed, delete its key — otherwise `in`, `distinct()` and `==` will all be wrong. The test `test_bag_equality_ignores_order` checks this.
- `__iter__` yields each item as many times as it occurs: a generator with two nested loops.
- `__eq__`: two bags are equal when their count dictionaries are equal. Return `NotImplemented` if other is not a `Bag`.
- `__repr__`: `sorted(self)` uses your `__iter__`; then an f-string with `!r` gives the quotes: `f'Bag({sorted(self)!r})'`.
- `__init__(self, items=())` — a tuple is immutable, so this default is safe. Initialise the attributes, then call `self.add` for each item.

Other hints:

- **2.** `word.strip('.,;:!?')` removes those characters from both ends only.
- **3.** `sorted(frequencies.items(), key=lambda pair: (-pair[1], pair[0]))` gives the order; then slice. Negating the count sorts high to low while the word still sorts A to Z.
- **4.** The empty matrix needs its own check: `matrix[0]` does not exist.
- **10.** `range(0, len(values), size)` gives the start of each chunk. Where does the `ValueError` get raised — when `chunks` is called, or when the first value is asked for? Read the test.

Part 10 — Bridge to this week's lecture: recursion

Lecture 03 is recursion, and the course exercises for it are in `dsa/recursion.py`, graded by `tests/test_recursion.py`. Everything here uses only what you already know — functions, `if`, and the call stack from Lab 02.

10.1 A function that calls itself

```
def countdown_rec(n):
    if n == 0:                                # base case: stop
        print('lift-off')
        return
    print(n)
    countdown_rec(n - 1)                      # recursive case: a SMALLER problem
```

```
>>> countdown_rec(3)
3
2
1
lift-off
```

Every recursive function needs both parts. The **base case** answers a problem small enough to answer directly. The **recursive case** makes the problem smaller and trusts the function to solve the smaller version.

10.2 Recursion that returns a value

The sum of a list: *the sum of an empty list is 0; otherwise it is the first item plus the sum of the rest.*

```
def total(values):
    if not values:
        return 0
    return values[0] + total(values[1:])
```

Trace `total([4, 7, 1])`. Each call waits, on the **call stack**, for the one it made:

```
total([4, 7, 1])
= 4 + total([7, 1])
    = 7 + total([1])
        = 1 + total([])
            = 0                                <- base case: the stack stops growing
        = 1 + 0 = 1                            <- and unwinds, top frame first
    = 7 + 1 = 8
= 4 + 8 = 12
```

Four frames were on the stack at the deepest point. For a list of length n , $n + 1$ frames: $O(n)$ stack space. (And, since `values[1:]` copies the list each time, $O(n^2)$ time — a better version passes an index instead of slicing. Noticing that is exactly the Week 2 habit.)

10.3 When it goes wrong

Forget the base case, or make a recursive case that does not get smaller, and Python stops you:

```
def forever(n):  
    return forever(n - 1)  
  
>>> forever(5)  
Traceback (most recent call last):  
  ...  
RecursionError: maximum recursion depth exceeded  
>>> import sys  
>>> sys.getrecursionlimit()  
1000
```

Raising the limit is almost never the fix. A wrong base case is.

10.4 Watch it in the debugger

Put `total` in a file, set a breakpoint on its first line (click in the gutter in VS Code), press **F5**, and step with **F11** (step into). Open the **Call Stack** panel: you will see one `total` frame per call, each with its own values. That picture is Lecture 03.

Then open `dsa/recursion.py` and start with `factorial`, `sum_digits` and `gcd`:

```
pytest tests/test_recursion.py -v -k "factorial or sum_digits or gcd"
```

Part 11 — Take-home practice (not graded)

1. Write `is_anagram(a, b)` three ways: sorting, a dictionary of counts, and your `Bag`. Which is $O(n \log n)$ and which $O(n)$?
2. Write a class `Matrix` with `__init__(rows)`, `__repr__`, `__eq__`, `__add__`, `__mul__` (matrix product) and a `transpose()` method. Raise `ValueError` for incompatible shapes.
3. Write a generator `fib_gen()` that yields Fibonacci numbers **forever**. Use it with `zip(range(10), fib_gen())` to print the first ten. Why is there no infinite loop?
4. Give your `Bag` an `__add__` that returns a new bag with the counts of both, and a `most_common(k)` method. Compare the results with `collections.Counter`.
5. Write `total(values, i=0)` from Part 10.2 again, without slicing. Time both versions for lists of 500 and 900 items. What did the slice cost?

Summary

Idea	The one line to keep
List costs	End is cheap (append, pop()), front is O(n) (insert(0), pop(0)), in is O(n).
sort vs sorted	sort() changes the list and returns None; sorted() returns a new list.
Stack / queue	A list is a fine stack and a bad queue. deque is the queue.
Comprehensions	[expr for x in xs if cond] — a loop that builds a list, in one line.
Tuples	Immutable; a comma makes one; unpack with a, b = t.
Sets	No duplicates; O(1) average in. set() is empty, {} is a dict.
Dicts	Key → value, O(1) average; insertion-ordered; get, setdefault, items.
Classes	Data in __init__ via self; methods take self; never a mutable class variable.
Special methods	__len__, __iter__, __eq__, __repr__ make your class feel built in.
Generators	yield pauses the function; lazy; the easiest way to write __iter__.
Recursion	A base case, and a recursive case that gets smaller.

Answers to the checkpoints

Checkpoint 1. (a) ['A', 'C'] (b) [1, 2, 2, 4] — i = 1 with j = 1, 2; then i = 2 with j = 1, 2. (c) [[1, 2], [2, 4]] — the inner comprehension builds one row per i. (d) [3] — only 'not' is longer than 2 characters.

Checkpoint 2.

```
[('a', 5), ('b', 2), ('c', 11)]
2
['b', 'a']
```

d.get('z', 10) gives the default 10, so 'c' is 1 + 10. Changing 'a' later keeps its position. 1, 1.0 and True are all equal and hash the same, so the set keeps just one of them, plus 'one': length 2. Both keys have length 1, so the key function gives a tie and the stable sort keeps dictionary order.

Checkpoint 3. 2 1 3 3 False. Each object has its own count, while Counter.total is one class variable updated through the class. a.total finds no instance attribute called total, so Python looks it up on the class. A new Counter has len 0, and with no __bool__, Python uses

`__len__` — so it is false.